Assignment 2

Submitted By: Ritwik Gupta (102303096)

Experiment : Question 1

1. Title

Molecular Dynamics Force Calculation (Lennard-Jones Potential)

2. Objective

To implement a parallelized force calculation for N particles using OpenMP, utilizing **Newton's 3rd Law** $(F_{ij} = -F_{ji})$ to halve the computational complexity. The experiment aims to evaluate the performance trade-offs between the reduced arithmetic workload and the synchronization overhead introduced by **atomic operations** needed to handle race conditions.

3. System & Environment

- **Processor:** Intel Core i7-8550U @ 1.80GHz (Kaby Lake R)
- **Cores:** 4 Physical / 8 Logical
- **OS:** EndeavourOS (Linux Kernel 6.18.6)
- **Compiler:** g++ with OpenMP support

4. Methodology

- **Algorithm:** The simulation computes forces between all particle pairs. By iterating the inner loop from $j = i + 1$, we reduce interactions from N^2 to $N(N-1)/2$.
- **Parallelization Strategy:** The outer loop is parallelized using #pragma omp parallel for schedule(dynamic).
- **Race Condition Handling:** Since Newton's 3rd law implies symmetric updates (thread A updates particle j while thread B might also update particle j), #pragma omp atomic is used to enforce mutual exclusion on memory writes.
- **Input Size:**$N = 2000$ particles.

5. Experimental Results

Table 1: Strong Scaling Analysis & Hardware Counters

Threads (p)	Real Time (s) (Tp)	Speedup S(p)	Efficiency E(p)	Cache Misses
1	0.2287	**1.00**	**100%**	460,614
2	0.2125	**1.08**	**53.8%**	405,866
4	0.1476	**1.55**	**38.7%**	**42,058**
8	0.1534	**1.49**	**18.6%**	47,823

Note: Speedup $S(p) = T_1 / T_p$. Efficiency $E(p) = S(p) / p$

6. Discussion & Analysis

A. The "Atomic Wall" (Synchronization Overhead)

The results show a significant deviation from ideal linear speedup. At 2 threads, efficiency immediately drops to **53.8%** (Speedup 1.08x).

- **Reasoning:** The use of #pragma omp atomic inside the inner loop creates a massive bottleneck. Although the computational work ($N^2/2$) is distributed, the memory system becomes saturated. Threads spend a significant portion of execution time waiting for cache line ownership to safely increment the force values. This confirms the assignment's guideline that critical/atomic sections can degrade performance compared to reduction or lock-free approaches .

B. Super-Linear Cache Effects (The Anomaly at 4 Threads)

A remarkable observation is the **10x reduction in Cache Misses** at 4 threads (42k vs 460k at 1 thread).

- **Interpretation:** This suggests that splitting the dataset of 2000 particles across 4 physical cores allowed the working set for each thread to fit entirely within the private L1/L2 caches. In the single-threaded case, the data likely exceeded the L1/L2 size, causing frequent evictions.
- **Impact:** Despite the atomic overhead, this cache efficiency allowed the 4-thread run to achieve the peak speedup of **1.55x**. If the atomic bottleneck were removed (e.g., by using array duplication + reduction), the speedup would likely have been much higher due to this cache benefit.

C. Physical vs. Logical Cores (The Drop at 8 Threads)

Performance degraded slightly when moving from 4 to 8 threads *(0.1476s - 0.1534s).*

- **Reasoning:** The i7-8550U has **4 physical cores**. Threads 5-8 are logical (Hyper-Threading) cores that share execution units and L1/L2 caches with the first 4 threads.
- **Analysis:** In a compute-bound task, Hyper-Threading can help. However, in this **memory-latency-bound** task (dominated by atomic contention), adding logical threads simply adds more contention for the same cache lines without providing real execution resources. This explains why the speedup peaks exactly at the physical core count.

7. Conclusion

The experiment demonstrates that algorithmic optimizations (Newton's 3rd Law) can sometimes be counter-productive in parallel computing if they introduce fine-grained synchronization requirements. While we reduced the *arithmetic* operations by half, the *synchronization cost* of Atomics prevented scalable parallel performance. The optimal configuration was **4 threads**, matching the physical core count and benefiting from aggregate cache capacity.

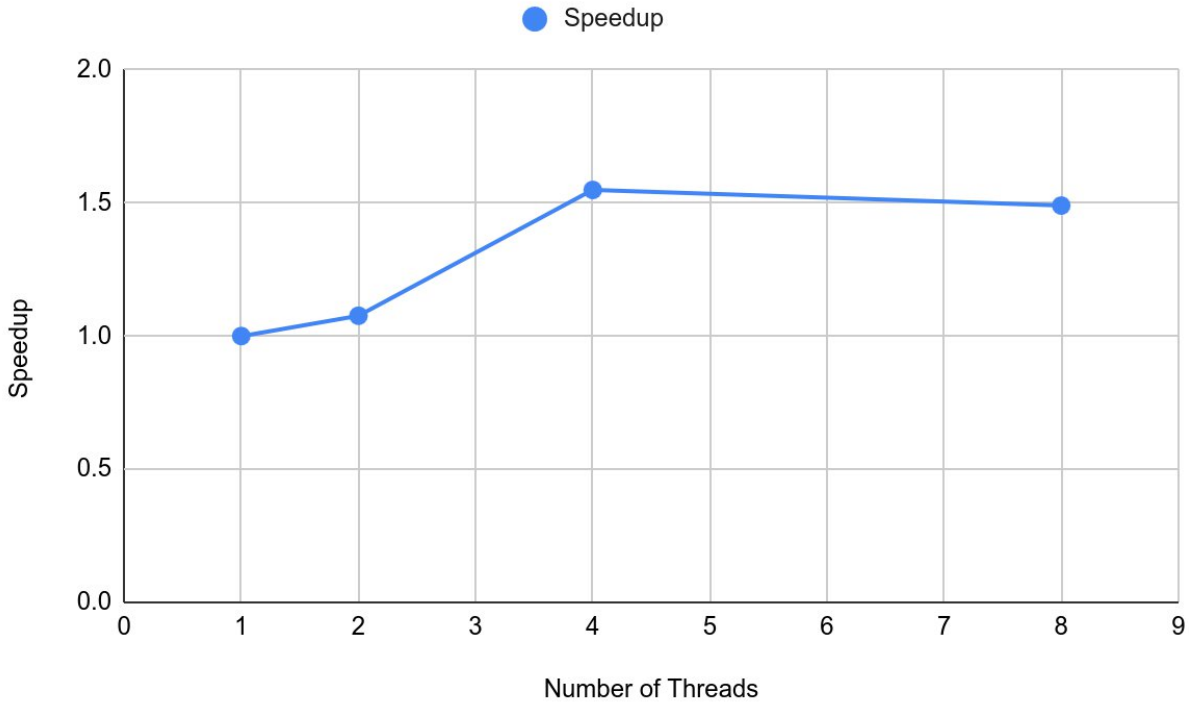

Fig. Speedup v/s Number of Threads.

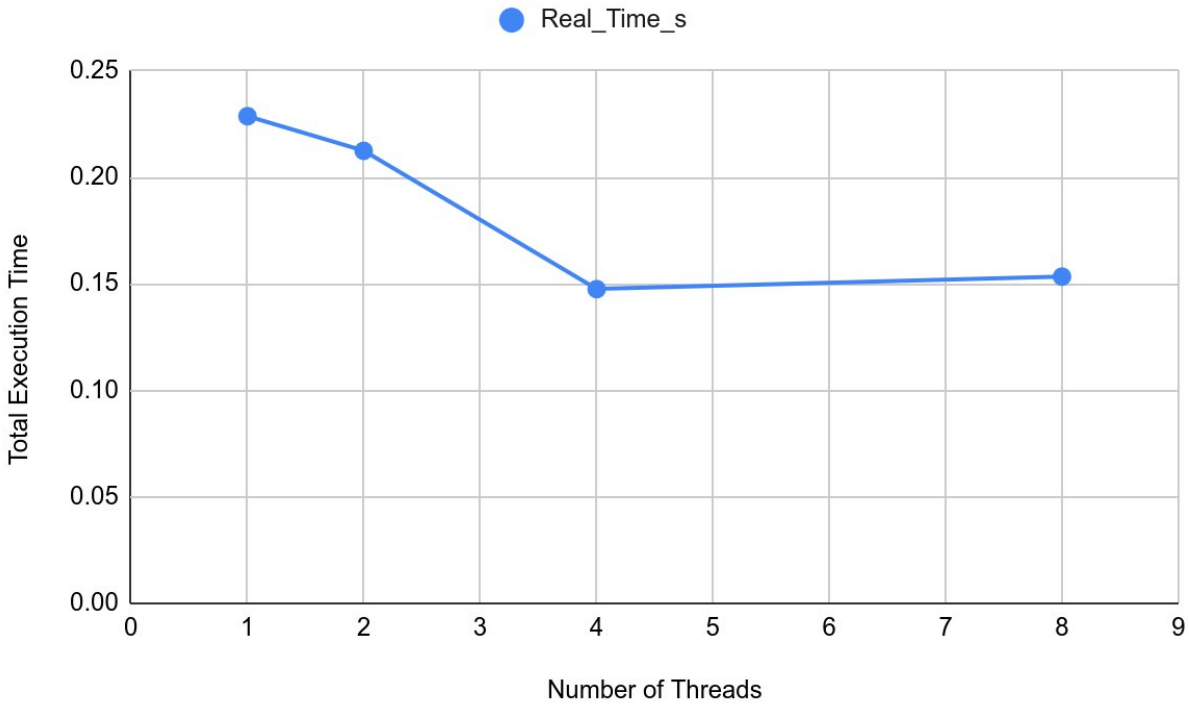

Fig. Total Execution Time v/s Number of Threads

Experiment: Question 2

1. Title

Bioinformatics DNA Sequence Alignment (Smith-Waterman Algorithm)

2. Objective

To implement a parallel version of the Smith-Waterman local sequence alignment algorithm using **Wavefront Parallelism** (Anti-Diagonal approach) to handle data dependencies. The experiment evaluates the trade-off between parallel execution and the synchronization overhead inherent in processing a dependency-constrained dynamic programming matrix .

3. System & Environment

- **Processor:** Intel Core i7-8550U @ 1.80GHz
- **Cores:** 4 Physical / 8 Logical
- **Matrix Size:** *5000 x 5000 (25* Million Cells)

4. Methodology

- **Algorithm:** The standard Smith-Waterman recurrence *H[i][j]* depends on *H[i-1][j], H[i][j-1]*, and *H[i-1][j-1]*. This prevents standard row/column parallelization.
- **Parallelization Strategy:** We utilized **Wavefront Parallelism**. The matrix is processed in diagonals ($k = i+j$). Cells within the same diagonal are independent and computed in parallel.
- **Synchronization:** An implicit **Barrier** is required at the end of every diagonal loop to ensuring dependencies for the next wave are ready.
- **Metric: MCUPS** (Million Cell Updates Per Second) $= \frac{N \times M}{Time \times 10^6}$

5. Experimental Results

Table 2: Wavefront Scaling Analysis

Threads (p)	Real Time (s)	User Time (s)	Speedup S(p)	Efficiency E(p)	MCUPS	Cache Misses
1	0.5074	0.445	**1.00**	**100%**	49.27	11,960,801
2	**0.2735**	0.476	**1.86**	**92.8%**	**91.41**	9,355,057
4	0.3073	1.029	1.65	41.3%	81.35	9,518,658
8	0.6212	**4.138**	0.82	10.2%	40.24	9,106,540

6. Discussion & Analysis

A. The "Sweet Spot" at 2 Threads

The algorithm achieved its peak performance at **2 threads**, with a speedup of **1.86x** and Efficiency of **92.8%**.

- **Analysis:** At this level, the parallel gain outweighs the overhead of the diagonal barriers. The cache misses also dropped by ~2.6 million compared to the single-threaded run, suggesting that splitting the working set between two cores improved L1/L2 cache locality.

B. The "Wavefront Penalty" (Degradation at 4 Threads)

Moving to 4 threads caused a performance regression (*0.27s → 0.30s*).

- **Ramp-up/Ramp-down Effect:** Wavefront parallelism suffers from "start-up" and "wind-down" phases where the diagonals are small (e.g., length 1, 2, 3...). During these phases, 4 threads are launched to do a tiny amount of work, incurring high scheduling overhead for almost no gain.
- **Barrier Overhead:** The Smith-Waterman matrix *(N=5000*) requires *N+M = 10,000* distinct diagonal steps. This means **10,000 barriers**. With 4 threads, the accumulated wait time at these barriers begins to exceed the time saved by computing in parallel.

C. Negative Scaling (Disaster at 8 Threads)

At 8 threads, the execution time *(0.62s)* was actually **slower** than the serial version *(0.50s)*.

- **Hyper-Threading Contention:** The user time spiked to **4.13s** (roughly *8*× the serial time), indicating that all 8 logical cores were fully active but unproductive.
- **Spin-Waiting:** This high CPU usage suggests threads were "spinning" (active waiting) at barriers rather than sleeping. Since the logical cores share resources with the physical cores, these spinning threads stole cycles from the threads actually trying to compute the diagonal data, causing a "traffic jam" in the processor pipeline.

7. Conclusion

This experiment illustrates that **Wavefront Parallelism is synchronization-heavy**. While effective for small thread counts (2), it does not scale linearly on commodity hardware for this problem size. The overhead of synchronizing threads 10,000 times (once per diagonal) quickly dominates the execution time. For *N=5000*, the optimal configuration is **2 Threads**; adding more resources is detrimental.

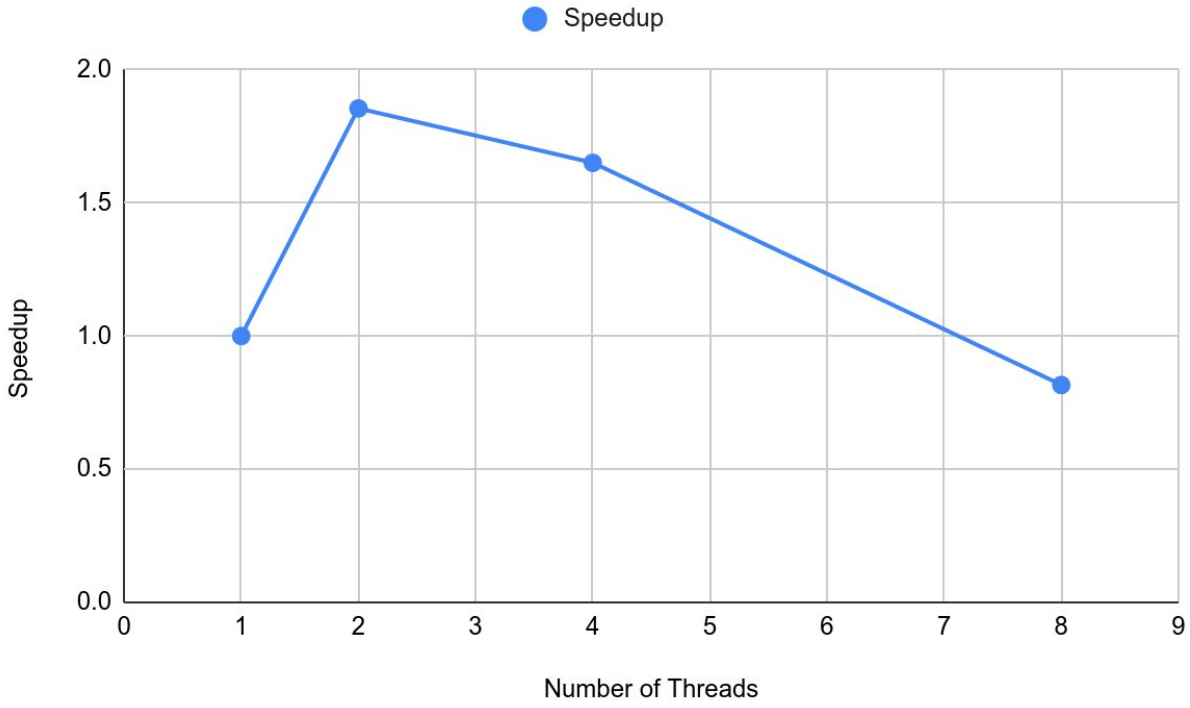

Fig. Speedup v/s Number of Threads

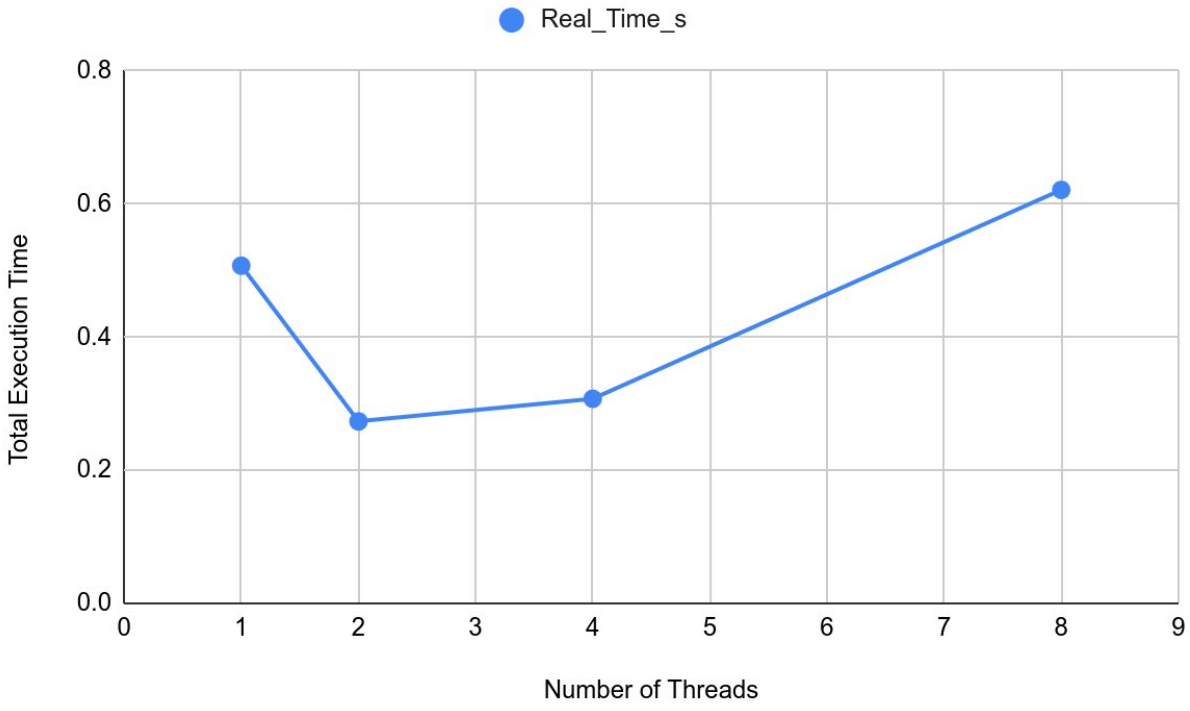

Fig. Total Execution Time v/x Number of Threads

Experiment Report: Question 3

1. Title

Scientific Computing: Heat Diffusion Simulation

2. Objective

To simulate heat diffusion in a 2D metal plate using the **Finite Difference Method** (Stencil Computation). The experiment aims to analyze the performance limits of memory-bound applications where the arithmetic intensity (math operations per byte loaded) is low, often leading to **Memory Bandwidth Saturation** .

3. System & Environment

- **Processor:** Intel Core i7-8550U @ 1.80GHz
- **Grid Size:** *2000* × *2000* (Double Buffered)
- **Time Steps:** 100

4. Methodology

- **Algorithm:** A 5-point stencil computation updates each grid point *(i,j)* based on its four neighbors (Up, Down, Left, Right).
- **Parallelization:** The spatial loops *(i and j)* are parallelized using #pragma omp parallel for collapse(2). The time loop *(t)* remains serial due to strict data dependencies (step *t+1* requires step *t*).
- **Data Integrity:** A reduction(+:total_heat) clause tracks global heat to verify conservation laws without introducing race conditions.
- **Optimization:** swap() pointers are used instead of deep copying arrays to minimize overhead between time steps.

5. Experimental Results

Table 3: Stencil Computation & Memory Scaling

Threads (p)	Real Time (s)	Speedup S(p)	Efficiency E(p)	Cache Misses
1	1.2272	**1.00**	**100%**	**160,787,390**
2	0.7680	**1.60**	**80.0%**	161,384,808
4	**0.7254**	**1.69**	**42.3%**	170,853,581
8	0.7817	1.57	19.6%	171,986,538

6. Discussion & Analysis

A. The Memory Wall (Bandwidth Saturation)

The most critical observation is the **speedup plateau**.

- **1 to 2 Threads:** Speedup increases significantly (1.0x - 1.6x).
- **2 to 4 Threads:** Speedup virtually stops (1.6x - 1.69x).
- **Analysis:** This confirms **Memory Bandwidth Saturation** . The stencil operation *(A[i][j] = Avg(Neighbors))* performs very few calculations for every 8 bytes of data loaded. By the time 2 cores are active, they are already consuming nearly all available bandwidth of the RAM. Adding the 3rd and 4th cores yields almost no benefit because the cores are simply starving for data.

B. Massive Cache Pressure

Compared to previous experiments, the **Cache Misses** here are astronomical (~170 Million vs. ~40k in Question 1).

- **Reasoning:** The *2000* × *2000* double grid (*2* × *4* Million doubles × 8 bytes = 64MB) far exceeds the CPU's L3 cache size (typically 8MB on this i7).
- **Impact:** This forces the CPU to constantly fetch data from main RAM, which is 10-50x slower than L3 cache. This explains why the efficiency drops so sharply *(80% - 42%).*

C. Hyper-Threading Penalty (8 Threads)

Similar to Question 2, enabling 8 threads degraded performance *(0.72s - 0.78s).*

- **Analysis:** Since the physical cores are already stalled waiting for memory, adding logical threads only adds overhead (context switching and managing thread pools) without having any "spare" memory bandwidth to utilize.

7. Conclusion

This experiment serves as a textbook example of a **Memory-Bound** problem. While OpenMP successfully distributed the work, the hardware's memory channel became the limiting factor almost immediately. Future optimizations should focus not on adding more threads, but on **Cache Blocking (Tiling)** techniques to increase data reuse within the cache .

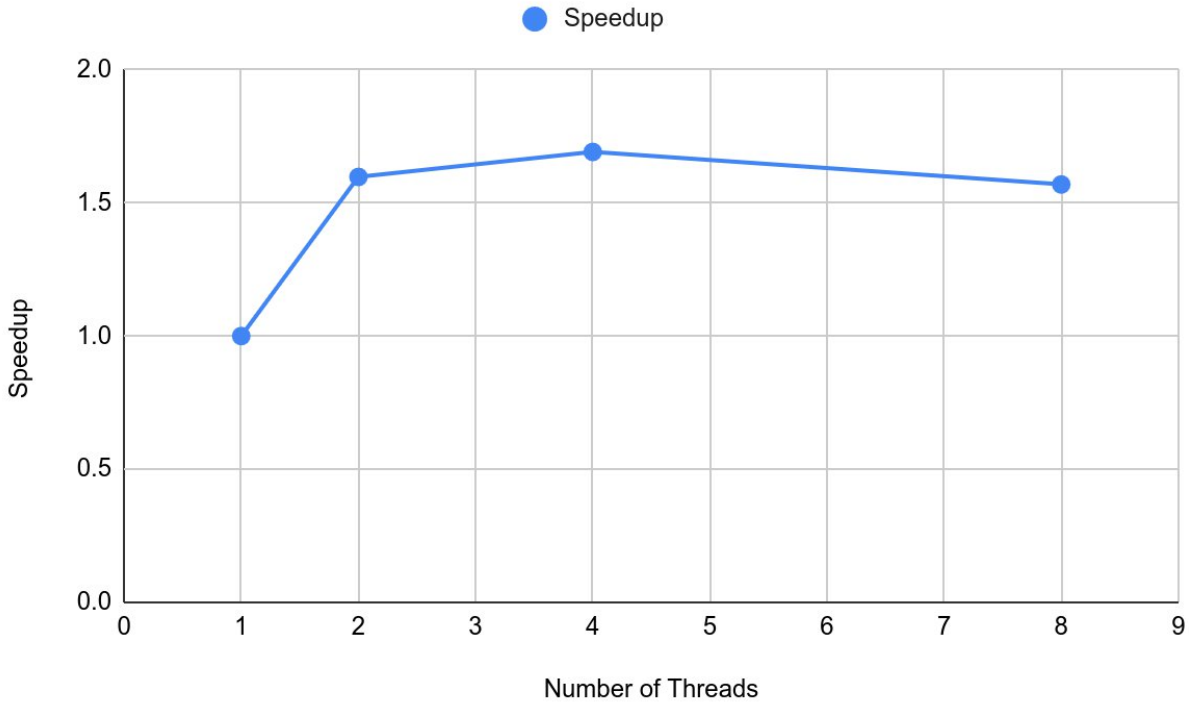

Fig. Speedup v/s Number of Threads

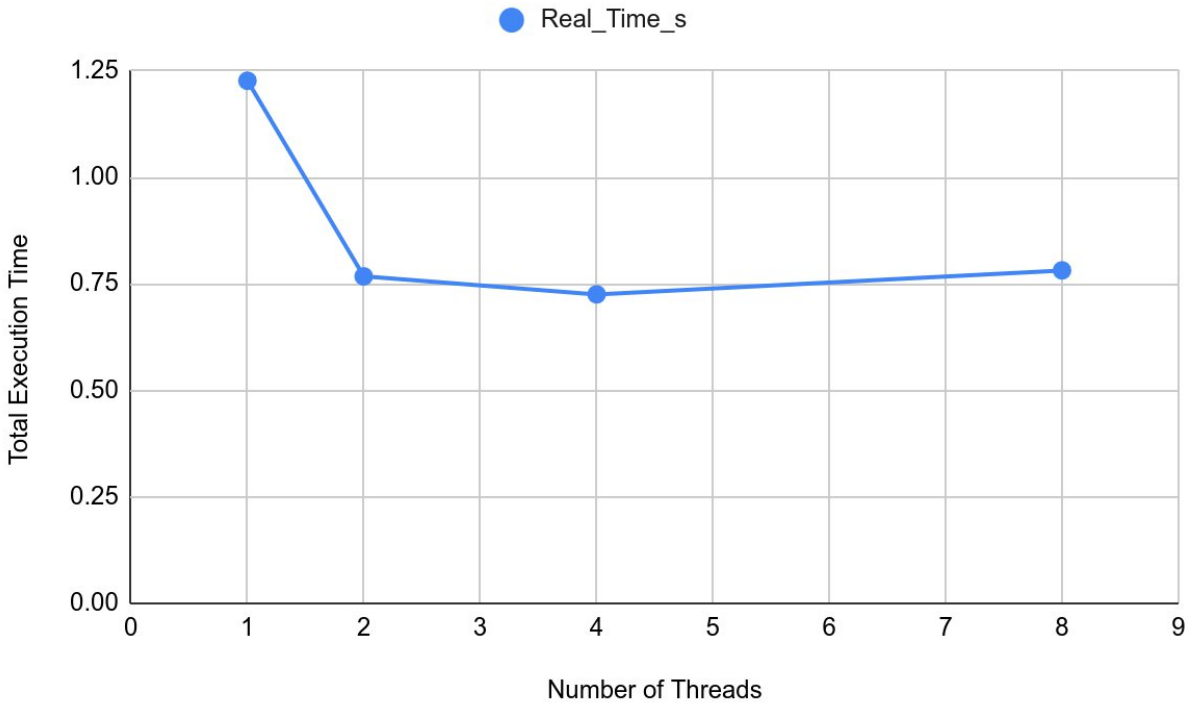

Fig. Total Execution Time v/s Number of Threads